

From Tables to Graph: A Unified Framework for Knowledge Graph Construction from Complex Relational Databases

Ziyu Zhao^{1,2}, Wei Liu², Tim French², Michael Stewart², Meng Chen³, and Feiyu Liao¹ (✉)

¹ Fujian Agriculture and Forest University, Fuzhou, China
zhaoziyu929@126.com, feiyu.liao@fafu.edu.cn

² The University of Western Australia, Perth, Australia
{wei.liu,tim.french}@uwa.edu.au, michael.Stewart@csiro.au

³ Oracle AI, Australia
chenmengdx@gmail.com

Abstract. Relational databases remain the dominant infrastructure for enterprise and scientific data, yet many emerging applications increasingly rely on graph-based knowledge representations for retrieval, reasoning, and analytics. Constructing knowledge graphs from complex relational databases is challenging because real-world data often contains incomplete constraints, heterogeneous schemas, and inconsistencies, while existing methods provide limited support for repair and semantic validation. We present Rel2KG, a unified framework that combines schema constraint-aware data repair, graph construction, and query-preserving migration. Rel2KG reconstructs missing keys, infers relational dependencies, resolves schema inconsistencies, and validates semantic preservation by comparing SQL results on source databases with Cypher results on generated property graphs. Experiments show that Rel2KG achieves strong semantic fidelity and scalability, with an average execution accuracy (EA) and a valid score (VS) of 97% and 88% on representative benchmarks, an EA of 88.43% and 87.75% on the Spider and KaggleD-BQA benchmarking datasets, respectively, and improved downstream semantic parsing, indicating that explicit graph structures reduce structural ambiguity and better support executable reasoning. The code is available at repository⁴.

Keywords: knowledge graph construction · data integration · SQL-to-Cypher translation · information and knowledge management.

1 Introduction

A knowledge graph (KG) represents information as entities and relationships explicitly modeled as nodes and edges [14]. Unlike relational databases, where relationships are encoded through primary–foreign key constraints and reconstructed at query time via multi-step joins, KGs externalize relational semantics directly

⁴ <https://github.com/22842219/Rel2KG>

as graph structures. In real-world enterprise environments, relational databases often serve as long-lived production systems that must increasingly interoperate with graph-based analytics and AI-driven applications. Enabling reliable and automated cross-representation migration is therefore essential for scalable data integration, knowledge management, and intelligent decision-support systems.

	Illustration 1	Illustration 2												
SQL Query	<pre>1 SELECT T1.department_id, T1.name, count(*) 2 FROM management AS T2 JOIN department AS T1 3 ON T1.department_id=T2.department_id 4 GROUP BY T1.department_id 5 HAVING count(*) > 1</pre>	<pre>1 SELECT count(*) FROM department 2 WHERE department_id NOT IN 3 (SELECT department_id FROM management)</pre>												
Cypher Query	<pre>1 MATCH ()-[T2:management]-(T1:department) 2 WITH T1.Department_ID AS Department_ID, 3 count(*) AS count, T1.Name AS Name 4 WHERE count > 1 5 RETURN Department_ID, Name, count</pre>	<pre>1 MATCH (d:department) 2 WHERE NOT (d:department)-[:management]-() 3 RETURN count(*)</pre>												
Execution Answer	<table><thead><tr><th>Return Field</th><th>Value</th></tr></thead><tbody><tr><td>Department_ID</td><td>2</td></tr><tr><td>Name</td><td>'Treasury'</td></tr><tr><td>count</td><td>2</td></tr></tbody></table>	Return Field	Value	Department_ID	2	Name	'Treasury'	count	2	<table><thead><tr><th>Return Field</th><th>Value</th></tr></thead><tbody><tr><td>count</td><td>11</td></tr></tbody></table>	Return Field	Value	count	11
Return Field	Value													
Department_ID	2													
Name	'Treasury'													
count	2													
Return Field	Value													
count	11													

Fig. 1. Comparison between SQL queries (top), corresponding Cypher queries (middle), and execution results (bottom).

The explicit relational structure of property graphs has enabled advanced applications such as intelligent question answering [20] and GraphRAG [11], which rely on structured reasoning over interconnected entities. Consequently, automatically transforming relational databases—the backbone of enterprise information systems—into knowledge graphs has become an important problem in applied data science. A robust solution must address heterogeneous schemas, implicit integrity constraints, data inconsistencies, and reproducibility challenges while maintaining semantic equivalence between relational and graph queries. As illustrated in Fig. 1, Cypher queries traverse explicit edges without requiring chained JOIN operations, whereas equivalent SQL queries compose multiple foreign-key joins to recover the same semantics. This structural explicitness reduces ambiguity and simplifies downstream reasoning and query generation.

Previous research in knowledge graph construction (KGC) follows mainly two paradigms, i.e. constructing KGs from text [31, 25, 3, 21] and from structured data [16, 26, 27, 24, 12]. Text-based approaches rely on entity linking [23], relation extraction [28], event extraction [6] and sequence-to-sequence generation models to derive graph structures from corpora [1]. In contrast, structured data approaches adopt Extract–Transform–Load (ETL) pipelines that migrate relational tables to graphs. However, current structured transformation methods face several limitations: (1) scalability issues when handling complex schema normalization and data cleansing [4]; (2) limited or absent mechanisms for constraint reconstruction or data repair during migration [24]; (3) insufficient reproducibility due to closed-source implementations, incomplete SQL semantic handling, and evaluation on small datasets or with flawed metrics (e.g., engine-dependent query time [12]).

To address these challenges, we present **Rel2KG**, an automated end-to-end framework for constructing property knowledge graphs from relational databases. Unlike existing migration approaches that primarily focus on schema transformation, Rel2KG emphasizes data quality assurance and semantic validation throughout the migration process. The framework integrates constraint-aware repair, graph construction, and SQL-to-Cypher translation within a unified pipeline. During migration, missing primary keys, incomplete foreign-key relationships, duplicate records, and domain-level naming conflicts are automatically identified and repaired. The resulting graph representation is validated through executable query equivalence rather than relying on schema-level inspection.

We evaluate Rel2KG on two large-scale public benchmarks, namely Spider [32] and KaggleDBQA [15]. Spider contains 5,693 complex SQL queries across 200 databases spanning 138 domains, while KaggleDBQA comprises real-world databases characterized by ambiguous schema naming and limited preprocessing. To assess whether relational semantics are preserved after migration, we introduce an execution-based validation strategy that compares the outputs of SQL queries executed on relational databases with the outputs of semantically equivalent Cypher queries executed on generated property graphs. This evaluation framework provides an engine-agnostic measure of semantic preservation and enables systematic assessment of migration correctness at scale.

Beyond migration correctness, we further investigate whether the graph representation produced by Rel2KG introduces a favourable inductive bias for downstream semantic parsing. Specifically, we compare Text-to-SQL and Text-to-Cypher under identical CodeT5 [33] configurations to examine whether explicit graph structures reduce representational ambiguity and improve executable query generation. This paper makes three contributions.

- **Constraint-aware relational-to-graph migration.** We propose an automated migration framework that reconstructs missing constraints, repairs schema inconsistencies, resolves domain-level naming conflicts, and converts relational databases into executable property knowledge graphs. Unlike conventional ETL-style migration tools, Rel2KG explicitly incorporates data repair and consistency checking into the transformation process.
- **Execution-equivalence-based semantic validation.** We introduce an execution-oriented evaluation framework that validates migration correctness through semantic equivalence between SQL and Cypher query execution results. This provides a practical and engine-independent mechanism for assessing information preservation beyond schema-level correspondence or runtime measurements.
- **Representation-level inductive bias analysis.** We demonstrate that graph representations generated by Rel2KG can improve downstream neural semantic parsing. Under identical CodeT5 configurations, Text-to-Cypher consistently outperforms Text-to-SQL, suggesting that explicit graph structures reduce hypothesis complexity and better align learned representations with execution semantics.

2 Related Work

2.1 Resource Description Framework (RDF) Oriented Mapping Frameworks

The foundational literature on relational-to-graph transformation established two World Wide Web consortium (WC) standards, namely direct mapping [2] and the relational database to RDF mapping language [7] (R2RML). Direct mapping provides an automatic mechanism for converting relational schemas and tuples into RDF graphs, whereas the R2RML enables customized mappings through logical tables, subject-predicate-object maps, and join conditions. The RDF mapping language further extends this paradigm to heterogeneous data sources beyond relational databases [10]. In parallel, ontology-based data access and virtual knowledge graph systems, such as Ontop [5], expose relational data through ontology-mediated conceptual layers and answer graph queries via query rewriting rather than physical graph construction.

2.2 Property-Graph-Oriented Migration and Query Translation

To support property graph databases, both academic and industrial efforts have investigated direct relational-to-property-graph transformation. The Neo4j ETL (Extract-Transform-Load) tool⁵ converts relational tables and foreign-key relationships into graph structures, providing a practical migration solution for graph database deployment. De Virgilio et al. [8, 9] proposed relational-to-graph transformation methods that preserve structural relationships during migration. Stoica et al. [27] further analyzed information preservation and query preservation in direct mappings from relational databases to property graphs. More recently, inspired by advances in pretrained large language models, graph construction research has increasingly shifted toward automated graph generation without manually designed mapping rules[30]. However, these methods are primarily designed for graph generation from unstructured text rather than structured relational databases, and therefore do not address relational integrity constraints, schema inconsistencies, or execution-level semantic equivalence. Consequently, existing migration approaches still provide limited support for repairing incomplete database schemas and validating whether migrated property graphs preserve the original relational semantics.

2.3 SQL-to-Cypher Translation

Prior work has explored query-preserving relational-to-graph (R2G) mapping. R2G translates relational data into graph structures and studies query translation from SQL into graph-oriented expressions [8, 9]. Stoica et al. [26, 27] analyze direct mapping from relational databases to property graphs and discuss

⁵ <https://neo4j.com/labs/etl-tool/>

information, query, and semantic preservation. SQL2Cypher further studies automated data and query migration from relational database management systems to graph database management systems [17]. Boudaoud et al. [4] provide a more formal analysis of direct relational-to-property-graph mapping, but their query translation focuses mainly on relatively simple SQL fragments such as `SELECT-FROM-WHERE` queries.

Recent benchmarks and datasets have also promoted research on natural-language-to-Cypher and graph query generation. Text2Cypher benchmarks and synthetic data generation methods provide resources for training and evaluating graph query generation models [22, 34]. CypherBench further contributes a large-scale benchmark for precise retrieval over modern knowledge graphs and includes RDF-to-property-graph conversion components [13]. These efforts are complementary to our work: they mainly focus on generating Cypher queries from natural language or existing graph data, whereas Rel2KG focuses on constructing executable property graphs from relational databases and validating SQL-to-Cypher equivalence.

3 Methodology

3.1 Problem Formulation

Given a source dataset denoted as $D_R = (\mathcal{S}, Q_{sql})$, our goal is to automatically construct a target dataset $D_G = (\mathcal{G}, Q_{cyp})$. Here, $\mathcal{S}$ represents a collection of relational databases, and $\mathcal{G}$ denotes the corresponding unified property knowledge graph. Q_{sql} is a set of SQL queries issued over $\mathcal{S}$, while Q_{cyp} is the corresponding set of Cypher queries intended to retrieve equivalent information from $\mathcal{G}$. The transformation consists of two main tasks: (1) Database Mapping: $\mathcal{S} \xrightarrow{\text{map}} \mathcal{G}$, and (2) Query Mapping: $Q_{sql} \xrightarrow{\text{map}} Q_{cyp}$.

First, we incrementally build the property knowledge graph $\mathcal{G}$ from the relational databases in $\mathcal{S}$ using a dedicated database mapping algorithm. Then, we convert SQL queries into their equivalent Cypher representations. To evaluate the effectiveness of both mappings, we assess data consistency between the two representations. Specifically, we measure how accurately the Cypher queries Q_{cyp} , when executed against the generated graph $\mathcal{G}$, reflect the results of original SQL queries Q_{sql} executed over relational source $\mathcal{S}$. For this, we employ the Execution Accuracy (EA) metric.

Each relational database $s \in \mathcal{S}$ is defined as a tuple $s = \langle T, C, V, PK, FK \rangle$, where: T is the set of table names, C is the set of column names, V contains the row values, PK denotes primary keys, and FK denotes foreign key relationships. We transform each s into a property knowledge graph represented as $\mathcal{G} = \langle \nu, \varepsilon, \mathcal{L}, \mathcal{T}, P_v, P_e \rangle$, where: ν is the set of graph nodes, ε is the set of graph edges, $\mathcal{L}$ represents node labels, $\mathcal{T}$ represents edge types, P_v denotes node properties, and P_e denotes edge properties. The detailed mapping procedure is described in the following sections.

3.2 Database Mapping

Metadata Analyzer To automate the mapping from relational databases to property graphs, we first analyze metadata—specifically primary key (PK) and foreign key (FK) constraints—to distinguish between entity tables and linking tables. We define each PK-FK pair as $\langle pk, fk \rangle$, where $pk \in PK$ and $fk \in FK$.

Algorithm 1: Rel2Graph Pseudocode

```

1 Input: A relational database schema dump  $s = \langle T, C, V, PK, FK \rangle$ .
2 Output: A property graph database  $\mathcal{G} = \langle \nu, \varepsilon, \mathcal{L}, \mathcal{T}, P_v, P_e \rangle$ .
3 RelDB2GraphBuilder( $s$ )
4  $ST_i = \langle t_i, c_i, v_i, pk_i, fk_i \rangle \leftarrow \{s\}$  ; // queue for each table
5 foreach  $ST_i$  do
6   if  $fk_i == \emptyset$  or  $len(fk_i) \neq 2$  or  $(len(fk_i) == 2 \wedge len(pk_i) == 1)$  then
7      $\mathcal{L} \leftarrow t_i$ 
8      $P_v \leftarrow \langle c_i, v_i \rangle$ 
9      $\nu = \{\mathcal{L}, P_v\}$  ; // Case one: entity tables are turned to graph
      nodes.
10    else if  $len(fk_i) == 2 \wedge (len(pk_i) \neq 1 \vee pk_i == \emptyset)$  then
11       $\mathcal{T} \leftarrow t_i$ 
12       $P_e \leftarrow \langle c_i, v_i \rangle$ 
13       $\varepsilon = \{\mathcal{T}, P_e\}$  ; // Case two: linking tables become graph edges.
14     $\mathcal{G} \leftarrow \nu \cup \varepsilon \cup \mathcal{L} \cup \mathcal{T} \cup P_v \cup P_e$ 
15 foreach  $fk_i \in ST_i$  do
16    $t_i \leftarrow ST_i$  ; // Get the table name  $t_i$  w.r.t.  $ST_i$ 
17    $t_j \leftarrow ST_j$ , WHERE  $fk_i == pk_j$  ; // Obtain the table name  $t_j$ 
      corresponding to  $ST_j$ , where the  $\langle fk_i, pk_j \rangle$  match the constraint
      present in the two entity tables.
18    $\varepsilon \leftarrow \{t_i\_HAS\_t_j\}$  ; // Case three: curated graph edges between
      two entity tables.
19    $P_e == \emptyset$ 
20 RETURN  $\mathcal{G}$ 

```

An entity table typically has a unique primary key and limited reliance on foreign keys. We classify a table as an entity table if it satisfies any of the observed conditions: (1) No foreign keys, i.e., $fk == \emptyset$; (2) Either one FK or more than two FKs, i.e., $len(fk) \neq 2$; (3) Two FKs and a single-column primary key, i.e., $len(fk) == 2 \wedge len(pk) == 1$. Each entity table is mapped to a graph node labelled with the table name. Each row becomes a node instance, and columns are converted to node properties (Algorithm 1), lines 6–9).

A table is identified as a linking table if it has exactly two foreign keys ($len(fk) = 2$) and either lacks a primary key ($pk = \emptyset$) or has a composite/multi-column primary key ($len(pk) \neq 1$). Linking tables are mapped to graph edges, with each row representing an edge, the table name as the edge type, and the columns as edge properties (Algorithm 1, lines 10–13).

If a table does not meet the criteria for either entity or linking tables, we create auxiliary edges of the form `_HAS_` to connect the relevant nodes. For example, as shown in Fig. 2, we generate an edge **Student_HAS_Enrolled_in** between nodes **Student** and **Enrolled_in**.

Fig. 2 demonstrates an example from `colledge_3.sqlite`. Tables **Faculty** and **Department** serve as entity tables with primary keys $\langle FACID \rangle$ and $\langle DNO \rangle$, respectively. Table **Member_of**, with two foreign keys $\langle FACID, DNO \rangle$, is clas-

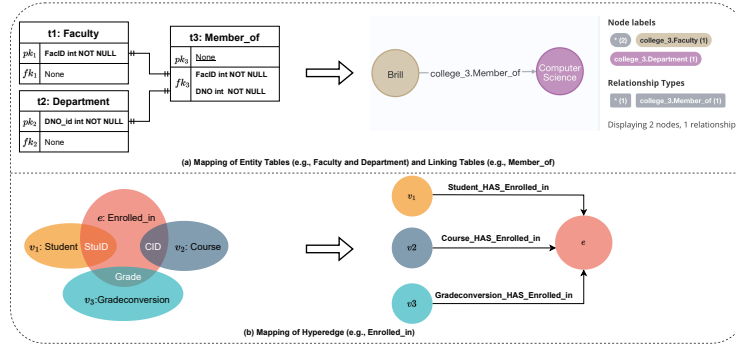


Fig. 2. Illustration of three mapping cases: (a) mapping entity tables **Faculty** and **Department** to nodes, (b) mapping the linking table **Member_of** to edges, and (c) converting a hyperedge **e** into three binary edges.

sified as a linking table and mapped to edges connecting the two entities (top right). For complex relationships involving more than two entities, such as a hyperedge **Enrolled_in** (bottom left), we convert the hyperedge into a node. Then we generate three edges from this node to the involved entities v_1 , v_2 , and v_3 . This workaround addresses graph databases such as Neo4j’s limitation in visualizing hyperedges while preserving semantic expressiveness.

Data Repairment Relational-to-graph schema conversion necessitates robust handling of structural inconsistencies that may disrupt query translation processes. Our automated metadata repair framework addresses four core challenges with validation performed across heterogeneous datasets, including industry-standard benchmarks. Constraint-based verification pipelines including anomaly detection on schema-semantic mismatches, ensuring relational semantics are bijectively preserved in graph structures.

(1) **Domain Identifier Collision Resolution.** We introduce namespace isolation via `DOMAIN.TABLE` labeling to eliminate cross-database identifier collisions. For example, both `SINGER.SQLITE` and `CONCERT_SINGER.SQLITE` contain a table named `singer`. Without explicit namespacing, queries may retrieve records from wrong databases. As shown in Table 1, querying `SINGER.SQLITE` incorrectly returns **Justin Brown**, who belongs to `CONCERT_SINGER.SQLITE`. Our `DOMAIN.TABLE` scheme ensures unambiguous table references across domains.

(2) **Primary Key Reconstruction.** For tables lacking explicit primary keys (10.8% of Spider tables), we infer candidate keys by analyzing foreign key dependencies in related tables. Columns referenced as foreign keys are promoted as primary key candidates. If no suitable column is identified, the table remains keyless. This strategy mitigates schema incompleteness while preserving graph conversion consistency.

(3) **Foreign Key Inference and Correction.** We address missing and erroneous constraints in three steps. First, implicit relationships are inferred from query patterns; for example, although the `STUDENTMATHSCORE.SQLITE`

Table 1. The examples for error analysis. ER is short for execution results.

Example 1	A nested query example.
NLQ:	What is the name of every singer that does not have any song?
SQL query:	SELECT Name FROM singer WHERE Singer_ID NOT IN (SELECT Singer_ID FROM song)
SQL query ER:	[[‘Alice Walton’], [‘Abigail Johnson’]]
Cypher query:	MATCH (si:singer) WHERE NOT (si:singer)-[:song] RETURN si.Name
Cypher query ER:	[[‘Justin Brown’], [‘Alice Walton’], [‘Abigail Johnson’]]
Example 2	An example that duplicate elements are dropped automatically.
NLQ:	Which skill is used in fixing the most number of faults? List the skill id and description.?
SQL query:	SELECT T1.skill_id, T1.skill_description FROM Skills AS T1 JOIN Skills_Required_To_Fix AS T2 ON T1.skill_id = T2.skill_id GROUP BY T1.skill_id ORDER BY count(*) DESC LIMIT 1
SQL query ER:	[[3, ‘TV, Video’]]
Cypher query:	MATCH (T1:‘assets_maintenance.Skills’)-[T2:‘assets_maintenance.Skills_Required_To_Fix’]-() WITH T1.skill_description AS skill_description, count(T1.skill_id) AS count, T1.skill_id AS skill_id RETURN skill_id, skill_description ORDER BY count DESC, LIMIT 1
Cypher query ER:	[[2, ‘Mechanical’]]

schema in KaggleDBQA declares no foreign keys, 75% of JOIN queries reveal latent associations. Second, referential integrity is validated by detecting and correcting misconfigured references. Third, to ensure update consistency, foreign keys are excluded from edge property sets, preventing inconsistencies when primary keys change.

(4) **Content Sanitization.** We perform automated deduplication, removing redundant records in 1.3% of Spider tables. For empty tables, null-safe placeholder injection preserves structural integrity during graph transformation.

3.3 Query Mapping

Graph data models are particularly efficient at pattern matching in highly connected datasets, while relational databases rely on costly JOIN ON operations using foreign keys. Consider the examples in Fig. 1. In Example 1, the JOIN ON operation is mapped directly into a Cypher MATCH clause, omitting the explicit reference to the foreign key *department_id*. In Example 2, w.r.t. **How many departments are led by heads who are not mentioned**, a nested SQL subquery is converted into a Cypher pattern within the WHERE clause. In both cases, Cypher eliminates the need for explicitly referencing foreign keys, as relationships are inherently encoded as edges.

Since Cypher is case-sensitive while SQL is not, we normalize all schema identifiers in SQL queries to match the case-sensitive graph schema, ensuring compatibility. The core SQL-to-Cypher translation involves mapping equivalent clauses and identifying graph patterns. We define a list of SQL keywords SQL_{key} ($SQL_{key} = [FROM, WHERE, SELECT, HAVING, GROUP BY, ORDER BY, LIMIT, OFFSET, UNION]$) and a corresponding list of Cypher keywords $Cypher_{key}$ ($Cypher_{key} = [MATCH, WHERE, RETURN, WITH...[AS], ORDER BY, LIMIT, OFFSET]$). The key mappings include: $FROM \rightarrow MATCH$, $SELECT \rightarrow RETURN$, $HAVING \rightarrow WITH...[AS]$, and $GROUP BY \rightarrow count()$. The remaining keywords are reused directly. JOIN operations are translated into graph patterns. Nested and grouped queries are connected using the WITH clause, passing results forward in a pipeline until all SQL clauses are translated.

4 Experimental Settings

4.1 Benchmarking Datasets

Although Spider[32] contains 169 databases and 8,034 SQL queries, the existing relational-to-graph migration baselines, such as Neo4j ETL⁵ and SQL2Cypher[17], require substantial manual configuration and schema adaptation when applied to all databases. Therefore, we construct a representative benchmark subset for comparative evaluation. The selected databases satisfy four criteria: (1) availability of corresponding SQL query workloads; (2) non-trivial relational schemas with multiple tables and foreign-key relationships; (3) high data completeness to minimize bias introduced by missing values; and (4) diversity in schema complexity, SQL complexity, and database scale.

Based on these criteria, we select four datasets from Spider as the main benchmark set as shown in Table 2. These databases collectively cover small and medium-sized schemas, large row volumes, dense foreign-key connectivity, complex SQL workloads, and varying graph sizes. Consequently, the benchmark provides a balanced evaluation environment for comparing graph construction quality, query preservation, and migration efficiency across different approaches, covering the majority of complexity levels observed in Spider.

Table 2. Benchmark database statistics and SQL query complexity.

Metric	store_1	college_3	hospital_1	tracking_software_problems
Tables	11	8	15	6
Foreign Keys	11	9	23	7
Rows	15,607	490	132	65
Queries	112	74	100	48
Characteristics	Dense schema	Association-heavy	High FK complexity	High SQL complexity
# Distinct SQL	56	37	50	24
# Joins	54	30	66	32
# Group By	28	12	20	8
# Nested	4	8	14	12
# Set Operations	4	6	12	8
# Complex SQL	14	6	12	14

4.2 Baseline Methods

Table 3. Feature comparison of relational-to-graph migration methods.

Feature	Neo4j ETL	SQL2Cypher [17]	Rel2KG (Ours)
Namespace Isolation	×	×	✓
Primary-Key Reconstruction	×	×	✓
Foreign-Key Repair	×	×	✓
SQL-to-Cypher Translation (basic)	×	✓	✓
Complex Query Support (nested/agg/ordering)	×	× (invalid outputs)	✓
Expression Normalization & Alias Resolution	×	×	✓
Semantic Query Preservation	×	×	✓

To evaluate the effectiveness of Rel2KG, we compare it against two representative baselines covering industrial practice and academic research. The feature comparison among methods is shown in Table 3.

Neo4j ETL-Style Tool⁵. Neo4j ETL is the official migration tool provided by Neo4j for transforming relational databases into property graphs. It automatically maps relational tables and foreign-key relationships into graph structures using predefined schema rules. As one of the most widely adopted industrial solutions for relational-to-graph migration, Neo4j ETL serves as a practical baseline for evaluating graph construction quality and migration efficiency.

SQL2Cypher. SQL2Cypher [17] is a representative academic approach that performs both relational schema migration and SQL-to-Cypher query translation. Unlike Neo4j ETL, SQL2Cypher explicitly considers query preservation during migration. Therefore, SQL2Cypher provides the closest methodological comparison. We applied a lightweight compatibility adapter to ensure executable evaluation while preserving the original mapping assumptions.

4.3 Evaluation Metrics

Execution Accuracy (EA) EA[19] is used to evaluate semantic preservation between relational and graph representations. Given a relational database and its migrated graph counterpart, EA measures whether the execution results of a Cypher query are identical to those of its semantically equivalent SQL query. Let R_{sql}^n and R_{cyp}^n denote the result sets returned by the n -th SQL query and Cypher query, respectively. EA is defined as:

$$EA = \frac{1}{N} \sum_{n=1}^N \mathbf{1}(R_{sql}^n, R_{cyp}^n) \quad (1)$$

where $\mathbf{1}(\cdot)$ denotes the indicator function:

$$\mathbf{1}(R_{sql}^n, R_{cyp}^n) = \begin{cases} 1, & \text{if } R_{sql}^n = R_{cyp}^n, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

where R_{sql}^n denotes the results by executing the Q_{sql}^n query, and R_{cyp}^n denotes the results by executing the semantically equivalent Cypher query Q_{cyp}^n .

Valid Score (VS) EA only considers successfully translated query pairs and does not account for SQL parsing failures. Consequently, a system may achieve a high EA score while covering only a small subset of the input query workload. To jointly evaluate translation correctness and coverage, we introduce VS, which penalizes parsing failures. A query is considered valid if it can be successfully parsed into a JSON-serializable syntax tree and translated into an executable Cypher query. VS is defined as:

$$VS = EA \times \frac{N}{N + N(F_{sql})} \quad (3)$$

where N is the number of successfully parsed examples and $N(F_{sql})$ is the number of failure examples. In order to provide a balanced measure that penalizes large discrepancies between structural correctness and query equivalence, harmonic accuracy (HA) is computed as the harmonic mean of EA and VS as follows.

$$HA = 2 \times \frac{EA \times VS}{EA + VS} \quad (4)$$

5 Experiment Results and Discussion

5.1 Semantic Preservation Performance

Research Question 1: How effectively does Rel2KG preserve relational semantics during graph construction and query migration?

Data Mapping Quality Table 4 compares the graph structures generated by different migration approaches. Neo4j ETL produces relatively compact graphs because it primarily relies on predefined foreign-key relationships and does not model many implicit semantic associations. In contrast, SQL2Cypher tends to generate substantially larger graphs, particularly on *store_1*, where nearly every relational record is transformed into graph entities and relationships. While this increases graph density, it also introduces redundant structures that do not necessarily contribute to query preservation.

Table 4. Data mapping statistics for each baseline and database.

Database	Neo4j ETL		SQL2Cypher		Rel2KG (Ours)	
	Nodes	Rels.	Nodes	Rels.	Nodes	Rels.
store_1	4,652	22,289	15,607	33,244	6,892	24,529
college_3	341	569	490	928	417	855
hospital_1	97	137	132	178	100	146
tracking_software_problems	65	105	65	105	65	105
Overall	5,155	23,100	16,294	34,455	7,474	25,635

Rel2KG generates graph structures of moderate size across all databases. Compared with Neo4j ETL, it captures richer semantic relationships through primary-key reconstruction, foreign-key inference, and relationship normalization. Compared with SQL2Cypher, it avoids excessive graph expansion by eliminating redundant mappings and preserving only semantically meaningful entities and relations. These results indicate that Rel2KG achieves a more balanced graph representation that better reflects the original relational semantics.

Query Preservation Performance Table 5 further evaluates whether the generated graphs preserve the semantics of the original SQL workloads. Rel2KG achieves the best overall performance, obtaining an average EA of 0.97, VS of 0.88, and HA of 0.92. The consistently high scores across all databases demonstrate that the generated graph schema maintains both structural correctness and query equivalence.

Table 5. Query preservation performance across benchmark databases.

Database	Neo4j ETL			SQL2Cypher			Rel2KG (Ours)		
	EA	VS	HA	EA	VS	HA	EA	VS	HA
store_1	0.60	0.05	0.10	0.81	0.23	0.36	0.96	0.93	0.95
college_3	0.33	0.24	0.28	1.00	0.08	0.15	1.00	0.97	0.99
hospital_1	0.08	0.06	0.07	1.00	0.06	0.11	0.98	0.86	0.91
tracking_software_problems	0.50	0.38	0.43	1.00	0.08	0.15	0.89	0.67	0.76
Overall	0.27	0.14	0.19	0.88	0.13	0.22	0.97	0.88	0.92

Neo4j ETL exhibits the weakest performance, with an overall HA of only 0.19. This result suggests that straightforward table-to-node conversion fails to preserve many relational semantics required for complex query execution. Although SQL2Cypher achieves relatively high EA scores on several databases, its VS values remain extremely low, resulting in an overall HA of only 0.22. The large discrepancy between EA and VS indicates that many translated queries are executable but semantically inconsistent with the original SQL results.

In contrast, Rel2KG maintains small gaps between EA and VS across most benchmarks, indicating that its schema-aware mapping strategy, alias resolution mechanism, and support for complex SQL constructs effectively preserve query semantics after migration. The superior HA score further confirms that Rel2KG simultaneously achieves high executability and high semantic fidelity, addressing a key limitation of existing relational-to-graph migration approaches.

5.2 Generalization Evaluation on Real-World Databases

Research Question 2: Can Rel2KG generalize to large-scale and noisy real-world databases while preserving semantic fidelity?

Table 6. Dataset statistics and complexity characteristics.

Dataset	#DB	#Queries	Avg Tables	Avg Rows	Characteristics
Spider	169	8,034	5.1	2K	Cross-domain, complex SQL
KaggleDBQA	8	272	2.3	280K	Real-world, noisy schemas

To evaluate the generalization capability of Rel2KG, we conduct experiments on the full Spider benchmark and KaggleDBQA [15], as summarized in Table 6. While Spider contains normalized schemas with complex SQL queries, KaggleDBQA comprises eight large-scale real-world databases featuring denormalized schemas, heterogeneous structures, and data inconsistencies. Evaluation on both datasets assesses whether Rel2KG can preserve relational semantics under both benchmark and real-world database settings.

Semantic Fidelity on Real-World Databases As shown in Table 7, Rel2KG achieves high execution accuracy on both datasets, with EA scores of 0.90 on

Spider and 0.87 on KaggleDBQA. The small performance gap demonstrates that the proposed unified migration framework preserves relational query semantics consistently across both benchmark databases and large-scale real-world schemas. Although KaggleDBQA contains noisier schemas, denormalized tables, and more heterogeneous data distributions, the execution accuracy decreases by only 0.03, indicating strong robustness to practical database characteristics. Spider achieves a higher VS (0.71) than KaggleDBQA (0.63), suggesting that real-world databases introduce additional challenges beyond semantic translation, including noisy schemas. Rel2KG obtains HA scores of 0.79 on Spider and

Table 7. Evaluation results of Rel2KG on Spider and KaggleDBQA.

Dataset	EA	VS	HA
Spider	0.90	0.71	0.79
KaggleDBQA	0.87	0.63	0.73

0.73 on KaggleDBQA. Compared with EA alone, HA provides a more comprehensive assessment of migration quality by penalizing methods that achieve high execution accuracy only on a limited subset of executable queries. The relatively high HA values on both datasets indicate that Rel2KG maintains both reliable semantic preservation and broad query applicability under heterogeneous database environments.

Migration Efficiency As shown in Fig. 3, the database migration is efficient, with an average runtime of approximately 5 seconds per database on the Spider benchmark. Only 11 databases require more than 10 seconds, and even the most complex case (*csu_1*) completes within 3 minutes. The runtime distribution highlights the scalability of the proposed Rel2KG framework.

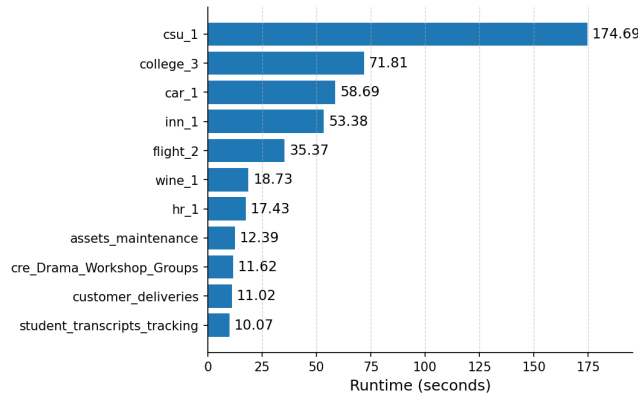


Fig. 3. Runtime distribution of the Rel2K process from Spider databases. The figure highlights the top 11 datasets with runtimes exceeding 10 seconds.

5.3 Case Study: Rel2KG as a Representation-Level Inductive Bias

Beyond schema transformation, we investigate whether Rel2KG introduces a representation-level inductive bias that improves semantic alignment between natural language questions and executable programs. We evaluate the hypothesis by comparing Text-to-SQL on Spider and Text-to-Cypher on CySpider under identical model configurations. We adopt CodeT5base[33], a transformer-based encoder-decoder pre-trained on large-scale code corpora with identifier-aware objectives, which has demonstrated strong performance on code generation tasks. We initialize from the released checkpoint and train on the task-specific training split, comparing full-parameter fine-tuning[29] and prefix-tuning[18] under identical optimization settings to ensure a controlled evaluation.

Table 8. Execution accuracy on the development set of Spider and CySpider using CodeT5 with prefix and fine-tuning, respectively.

Tuning Strategy	Text-to-SQL (Spider)	Text-to-Cypher (CySpider)
CodeT5 _{base} (prefix tune)	64.78	66.61
CodeT5 _{base} (fine tune)	68.11	71.43

As shown in Table 8, full fine-tuning consistently outperforms prefix-tuning on both benchmarks. On Spider, execution accuracy increases from 64.78% to 68.11%, while on CySpider it improves from 66.61% to 71.43%. Under the same model architecture and training settings, Text-to-Cypher further surpasses Text-to-SQL, indicating that the performance gain originates from the underlying representation rather than model capacity. Unlike relational databases, where semantic relations are implicitly encoded through foreign-key joins, the property graph generated by Rel2KG represents relations explicitly as edges, reducing structural ambiguity and simplifying query composition. The results suggest that Rel2KG preserves relational semantics while introducing a favourable inductive bias for semantic parsing, thereby improving executable query generation.

6 Conclusion

This paper presented Rel2KG, an automated framework for transforming relational databases into property knowledge graphs while preserving relational semantics. By integrating constraint-aware schema mapping, data repair, graph construction, and SQL-to-Cypher translation, Rel2KG enables accurate and scalable relational-to-graph migration across heterogeneous databases. Experimental results on benchmark and real-world datasets demonstrate that Rel2KG consistently outperforms existing approaches in query preservation, achieving high semantic fidelity while maintaining efficient transformation performance. Furthermore, the generated graph representation improves downstream semantic parsing accuracy by converting implicit relational joins into explicit graph relationships, thereby reducing structural ambiguity and facilitating reasoning over complex data. These findings show that Rel2KG provides a robust foundation for graph-based data integration and knowledge-driven applications.

References

1. Agarwal, O., Kale, M., Ge, H., Shakeri, S., Al-Rfou, R.: Machine translation aided bilingual data-to-text generation and semantic parsing. In: Proceedings of the 3rd international workshop on natural language generation from the semantic web (WebNLG+). pp. 125–130 (2020)
2. Arenas, M., Bertails, A., Prud’hommeaux, E., Sequeda, J.F.: A direct mapping of relational data to RDF. Tech. rep., World Wide Web Consortium (2012)
3. Bikaun, T., Stewart, M., Liu, W.: Quickgraph: A rapid annotation tool for knowledge graph extraction from technical text. In: Proc. ACL (System Demonstrations). pp. 270–278 (2022)
4. Boudaoud, A., Mahfoud, H., Chikh, A.: Towards a complete direct mapping from relational databases to property graphs. In: International Conference on Model and Data Engineering. pp. 222–235 (2022)
5. Calvanese, D., Cogrel, B., Komla-Ebri, S., Kontchakov, R., Lanti, D., Rezk, M., Rodriguez-Muro, M., Xiao, G.: Ontop: Answering sparql queries over relational databases. *Semantic Web* **8**(3), 471–487 (2017)
6. Chen, Y., Xu, L., Liu, K., Zeng, D., Zhao, J.: Event extraction via dynamic multi-pooling convolutional neural networks. In: ACL-IJCNLP. pp. 167–176 (2015)
7. Das, S., Sundara, S., Cyganiak, R.: R2rml: Rdb to rdf mapping language. W3c recommendation, World Wide Web Consortium (2012)
8. De Virgilio, R., Maccioni, A., Torlone, R.: Converting relational to graph databases. In: First International Workshop on Graph Data Management Experiences and Systems. pp. 1–6 (2013)
9. De Virgilio, R., Maccioni, A., Torlone, R.: R2g: A tool for migrating relations to graphs. In: Proc. EDBT. pp. 640–643 (2014)
10. Dimou, A., Vander Sande, M., Colpaert, P., Verborgh, R., Mannens, E., Van de Walle, R.: Rml: A generic language for integrated rdf mappings of heterogeneous data. In: Proceedings of the 7th Workshop on Linked Data on the Web (2014)
11. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph rag approach to query-focused summarization (2025)
12. Feng, H., Huang, M.: An approach to converting relational database to graph database: from mysql to neo4j. In: 2022 IEEE 2nd International Conference on Power, Electronics and Computer Applications. pp. 674–680. IEEE (2022)
13. Feng, Y., Papicchio, S., Rahman, S.: CypherBench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1). pp. 8184–8201 (2025)
14. Jiang, X., Xu, C., Shen, Y., Sun, X., Tang, L., Wang, S., Chen, Z., Wang, Y., Guo, J.: On the evolution of knowledge graphs: A survey and perspective (2025)
15. Lee, C.H., Polozov, O., Richardson, M.: Kaggledbqa: Realistic evaluation of text-to-sql parsers. In: Proc. ACL-IJCNLP. pp. 2261–2273 (2021)
16. Lee, S., Park, B.H., Lim, S.H., Shankar, M.: Table2graph: A scalable graph construction from relational tables using map-reduce. In: IEEE 1st International Conference on Big Data Computing Service and Applications. pp. 294–301 (2015)
17. Li, S., Yang, Z., Zhang, X., Zhang, W., Lin, X.: SQL2Cypher: Automated data and query migration from rdbms to gdbms. In: International Conference on Web Information Systems Engineering (2021)

18. Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the ACL and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 4582–4597 (2021)
19. Lin, X.V., Socher, R., Xiong, C.: Bridging textual and tabular data for cross-domain Text-to-SQL semantic parsing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (2020)
20. LlamaIndex Team: Introducing the property graph index: A powerful new way to build knowledge graphs with LLMs (2023), accessed: 2024-10-24
21. Melnyk, I., Dognin, P., Das, P.: Knowledge graph generation from text. In: Findings of EMNLP 2022. pp. 1610–1622 (2022)
22. Ozsoy, M.G., Messallem, L., Besga, J., Minneci, G.: Text2Cypher: Bridging natural language and graph databases. In: Proceedings of the Workshop on Generative AI and Knowledge Graphs (GenAIK). pp. 100–108. International Committee on Computational Linguistics, Abu Dhabi, UAE (Jan 2025)
23. Paolini, G., Athiwaratkun, B., Krone, J., Ma, J., Achille, A., Anubhai, R., Santos, C.N.d., Xiang, B., Soatto, S.: Structured prediction as translation between augmented natural languages. arXiv preprint arXiv:2101.05779 (2021)
24. Putrama, I.M., Martinek, P.: An automated graph construction approach from relational databases to neo4j. In: Proc. CINTI-MACRo. pp. 131–136 (2022)
25. Stewart, M., Liu, W., Cardell-Oliver, R.: Redcoat: A collaborative annotation tool for hierarchical entity typing. In: Proc. EMNLP-IJCNLP (System Demonstrations). pp. 193–198 (2019)
26. Stoica, R., Fletcher, G., Sequeda, J.F.: On directly mapping relational databases to property graphs. In: 13th Alberto Mendelzon International Workshop on Foundations of Data Management, AMW 2019. p. 06. CEUR-WS. org (2019)
27. Stoica, R.A.: R2PG-DM: a direct mapping from relational databases to property graphs. Ph.D. thesis, Eindhoven University of Technology (2019)
28. Wei, Z., Su, J., Wang, Y., Tian, Y., Chang, Y.: A novel cascade binary tagging framework for relational triple extraction. arXiv preprint arXiv:1909.03227 (2019)
29. Xie, T., Wu, C.H., Shi, P., Zhong, R., Scholak, T., Yasunaga, M., Wu, C.S., Zhong, M., Yin, P., Wang, S.I., Zhong, V., Wang, B., Li, C., Boyle, C., Ni, A., Yao, Z., Radev, D., Xiong, C., Kong, L., Zhang, R., Smith, N.A., Zettlemoyer, L., Yu, T.: UnifiedSKG: Unifying and multi-tasking structured knowledge grounding with text-to-text language models. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing. pp. 602–631 (Dec 2022)
30. Ye, H., Zhang, N., Chen, H., Chen, H.: Generative knowledge graph construction: A review. In: Proceedings of the 2022 Conference on EMNLP. pp. 1–17. Association for Computational Linguistics (Dec 2022)
31. You, J., Ying, R., Ren, X., Hamilton, W., Leskovec, J.: Graphrnn: Generating realistic graphs with deep auto-regressive models. In: International conference on machine learning. pp. 5708–5717 (2018)
32. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: EMNLP. pp. 3911–3921 (Oct-Nov 2018)
33. Yue Wang, Weishi Wang, S.J.S.C.H.: Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: EMNLP (2021)
34. Zhong, Z., Zhong, L., Sun, Z., Jin, Q., Qin, Z., Zhang, X.: Synthet2c: Generating synthetic data for fine-tuning large language models on the text2cypher task. In: Proc. COLING. pp. 672–692 (2025)